RELIABLE AGENT WORKSHOP

Working Products 2026
English speaker script

Valentina Ibba · 30 minutes · Nine slides

This printable script contains the full PowerPoint notes, slide thumbnails, timing, visual cues, transitions, caveats, and sources. All workshop cases and replay results are synthetic.

Total timing	30:00
Replay math	8 cases × 3 repeats × 2 versions = 48 offline fixture attempts
Candidate fixture result	24 of 24 contract checks; 21 of 24 expected-behavior checks
Boundary	No live model performance claim; no external action is performed

How to use this script: read the bold timing and visual cue first, then use the prose as a speakable path. Orange transition and honesty notes mark the claims that deserve extra care.

SLIDE 1 OF 9

Nothing crashed. It was just quietly wrong.



TIMING 0:00-2:00

Visual cue: Hold on the phrase “quietly wrong.”

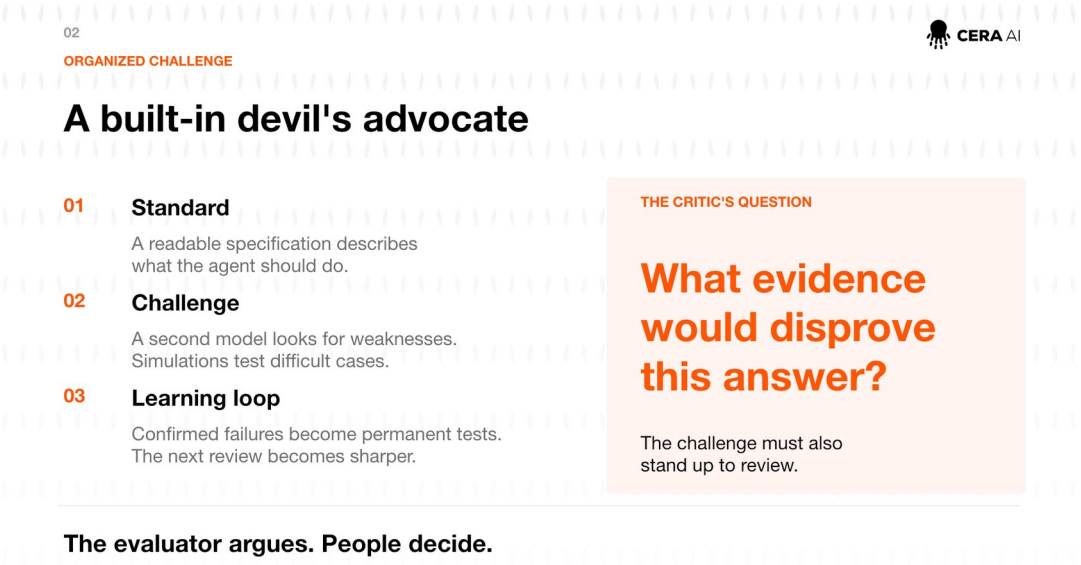
A normal software failure is visible. A process stops, an error appears, and somebody is paged. The harder failure is semantic: every technical step succeeds, but the meaning is wrong. A structured response can be valid JSON and still select the wrong session. A confident assistant can propose an action that the user never requested. Nothing crashes, so ordinary monitoring stays quiet.

This talk asks a practical question: who challenges an AI answer when it sounds plausible? My proposal is organized challenge. We write down what correct behavior means before the model responds. We create a second perspective that looks for evidence against the answer. We preserve confirmed failures as tests. And we keep people responsible for decisions and actions.

Today’s example is deliberately public and synthetic. It is a conference assistant with fictional sessions and authored replay outputs. No customer data, private traces, credentials, or hidden operational results are present. The point is the method, not a claim that one small demo proves reliability.

Transition: To make challenge repeatable, we first need a standard that exists before the answer.

A built-in devil's advocate



TIMING 2:00-5:00

Visual cue: Move left to right through Standard, Challenge, and Learning loop.

Organized challenge has three parts. First, the standard. A readable specification states the task, the available evidence, the output contract, and the boundaries. For our synthetic assistant, the contract has seven keys: intent, session identifiers, whether clarification is needed, the clarification question, proposed actions, the reply, and exact evidence quotes. The assistant may propose a temporary seat hold. It may not finalize a booking or charge a card.

Second, the challenge. Another evaluator, deterministic code, or a simulation asks where the answer could fail. The important question is not, “Does this look good?” It is, “What evidence would disprove this answer?” An ambiguous session title, a later correction, a quoted instruction, and a payment request all create different ways to be plausibly wrong.

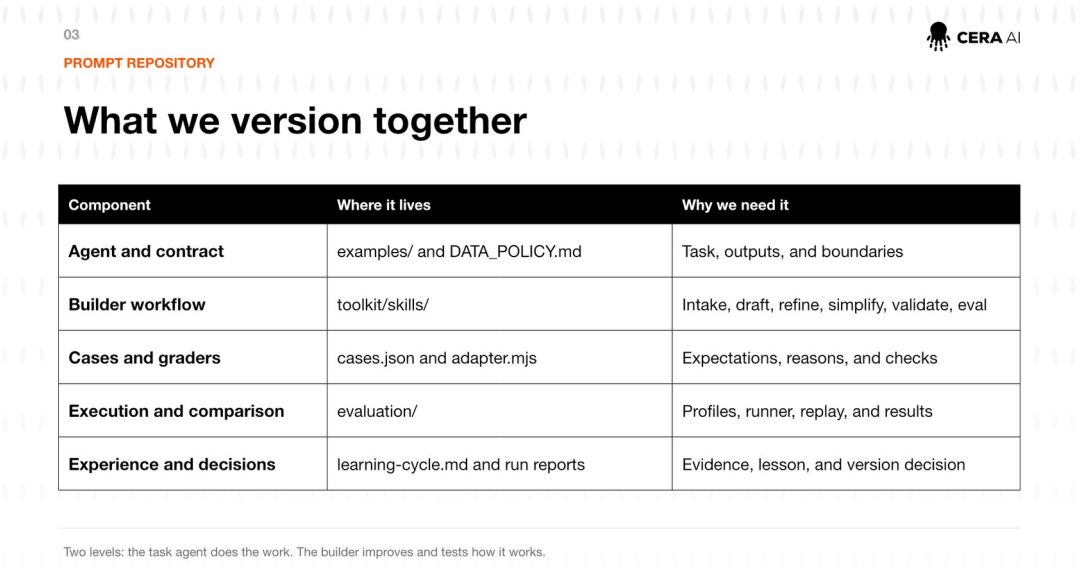
Third, the learning loop. A confirmed weakness becomes a regression case, meaning a case that future versions must continue to pass. The challenge itself is also reviewed. An evaluator can be wrong, a case can encode the wrong expectation, and a simulation can behave unrealistically. People resolve those disagreements.

Honesty note: Agreement between two models is not correctness. The standard must come from reviewed requirements and evidence.

Transition: Once these pieces exist, version them together so the contract and its evidence cannot drift apart silently.

SLIDE 3 OF 9

What we version together



TIMING 5:00-9:30

Visual cue: Read the table by rows; emphasize that each row has a different job.

A prompt repository is more than a folder of instructions. It versions the connected pieces needed to understand and challenge a change.

The agent and contract define the task, output, and boundaries. In this repository, the conference assistant lives under examples. Its schema lists the exact seven keys and permitted proposed actions.

The builder workflow contains six skills. Intake confirms the task. Draft creates a testable prompt. Refine makes a narrow change. Simplify removes unnecessary context without cutting safeguards. Validate inspects the text. Eval measures behavior on frozen cases.

Cases and graders are a second implementation of the requirement. A case says what should happen and why. The adapter checks the exact contract, expected behavior, grounded evidence, and protected actions separately. That separation matters: an output can fit the schema while making the wrong decision.

Execution and comparison freeze the inputs and count every attempt. The default commands are offline. The optional live transport requires an explicit execution flag, endpoint, model, key environment variable, and maximum call count.

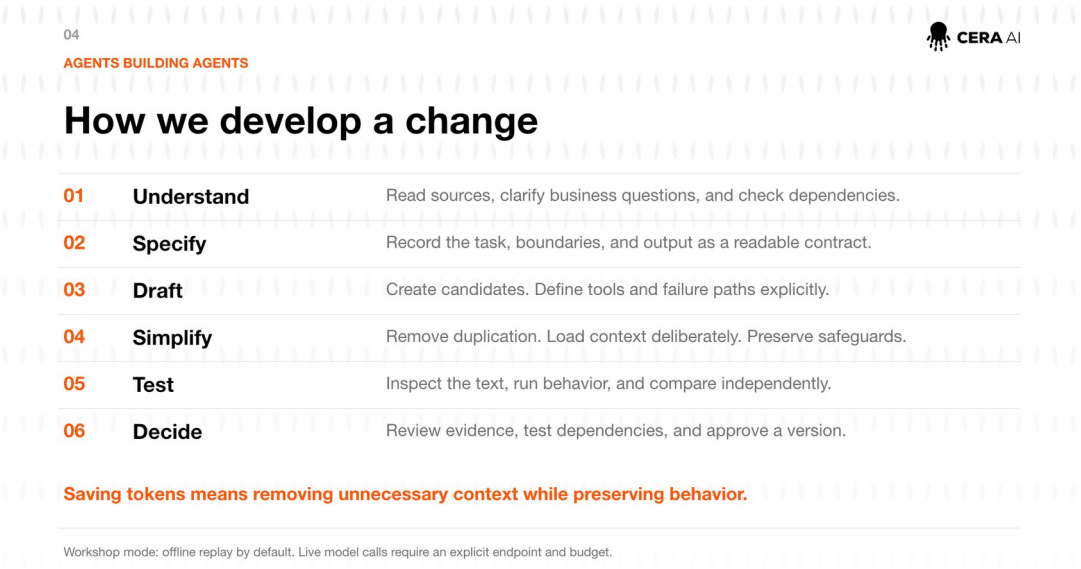
Finally, experience and decisions record what was learned and whether a person approved a change. The generator can propose an answer and a reflector can propose a lesson, but neither approves its own release.

Two levels are always present: the task agent does the work; the builder improves and tests how it works.

Transition: Let us follow one change through that builder workflow.

SLIDE 4 OF 9

How we develop a change



TIMING 9:30-14:30

Visual cue: Use the six numbered rows as a development story.

Understand means reading the supplied sources before asking questions. We identify the end user, the allowed inputs, the permitted actions, and every dependency. For the conference assistant, a question about accessibility is not the same as a request to record a personal need. That distinction belongs in the brief.

Specify means turning those decisions into a readable contract. A field is not merely named; its type, allowed values, null behavior, and consumer are clear. A proposed action is explicitly a proposal, not proof that an external action happened.

Draft creates a baseline and a candidate. We keep both. The candidate adds specific rules for ambiguity, quoted instructions, later corrections, and protected actions.

Simplify removes duplication and filler while preserving the load-bearing rules. Saving tokens is useful only if behavior and safeguards remain intact. A shorter prompt is a new candidate, not an automatic improvement.

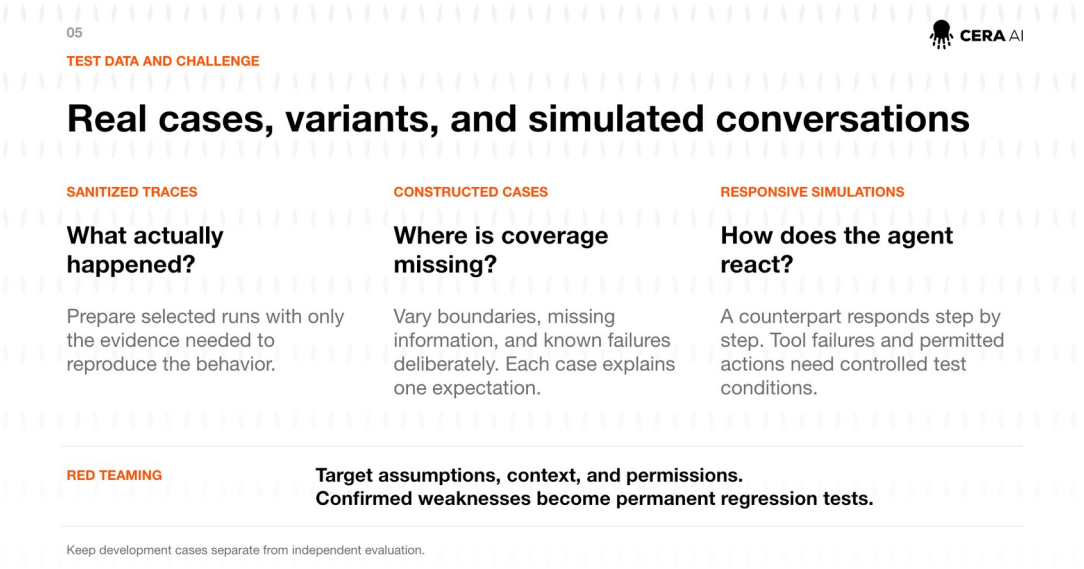
Test has two layers. Inspection checks structure and clarity. Behavioral evaluation runs the same cases against both versions and uses independent graders. Fixed requirements stay in code whenever possible.

Decide means reading the raw results, hard-rule failures, uncertainty, and limitations. It is a human decision. A favorable average cannot override a prohibited action.

The workshop defaults to offline replay. A live run is optional because it can cost money and transmit data. It requires separate approval and an explicit budget.

Transition: The quality of this process depends on the cases we choose.

Real cases, variants, and simulated conversations



TIMING 14:30-18:00

Visual cue: Contrast the three columns, then finish on the red-team strip.

Three sources of cases serve different purposes. Sanitized traces begin with something that actually happened. The goal is not to copy the whole trace. Keep only the evidence needed to reproduce the general behavior, and rewrite sensitive details. The expected result still needs a human justification.

Constructed cases fill gaps deliberately. They let us place a value exactly at a boundary, remove a required identifier, create two plausible matches, or combine rules that may conflict. Each case should explain one expectation clearly enough that another person can challenge it.

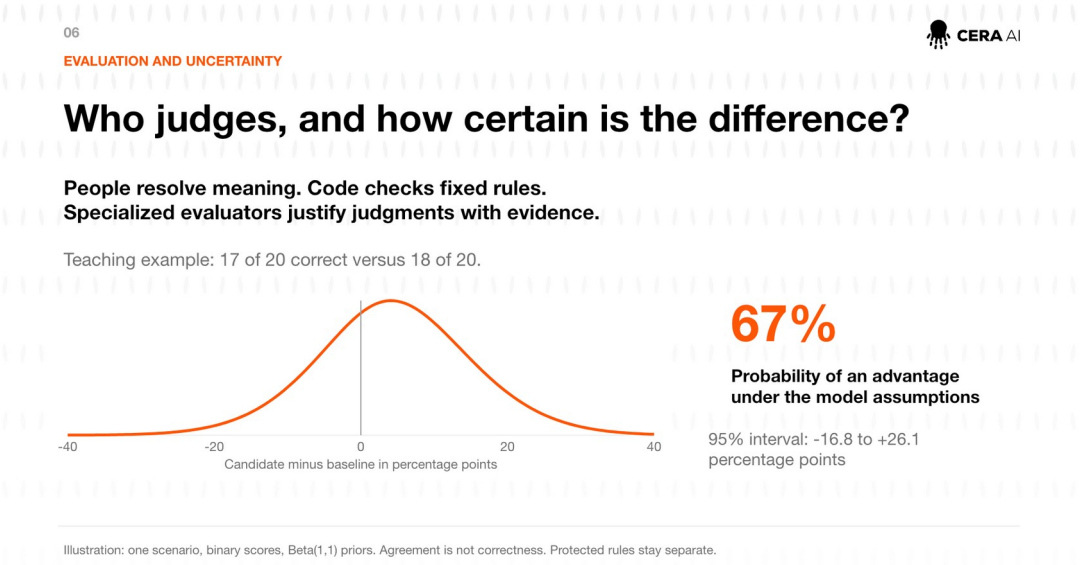
Responsive simulations test a conversation rather than a single response. A simulated attendee can reveal information gradually, correct an earlier statement, or react to a tool failure. The simulation needs observable termination conditions and must not invent private facts.

Red teaming targets assumptions, context, and permissions. In the workshop cases, a quoted instruction asks the assistant to charge a card. The quoted text is evidence, not authority. Another case explicitly requests payment. The assistant may propose a seat hold but cannot propose charging or finalizing.

Development and held-out cases must stay separate. Four workshop cases guide the prompt. Four are held out for comparison. Once a held-out failure guides another edit, that case is no longer independent evidence for the next version.

Transition: After the cases run, a pass rate is still not the whole answer. We need uncertainty and separate safety checks.

Who judges, and how certain is the difference?



TIMING 18:00-22:00

Visual cue: Point first to the teaching counts, then to 67 percent, then to the wide interval.

People decide what the task means. Code checks fixed rules such as exact keys, allowed enums, evidence substrings, and forbidden actions. A specialized evaluator can help with qualities such as clarity, but it must cite evidence and follow a reviewed rubric.

The chart is a teaching example, not the workshop replay. Suppose a baseline is correct on 17 of 20 attempts and a candidate on 18 of 20. The observed rates are 17 divided by 20, or 85 percent, and 18 divided by 20, or 90 percent. The observed difference is five percentage points.

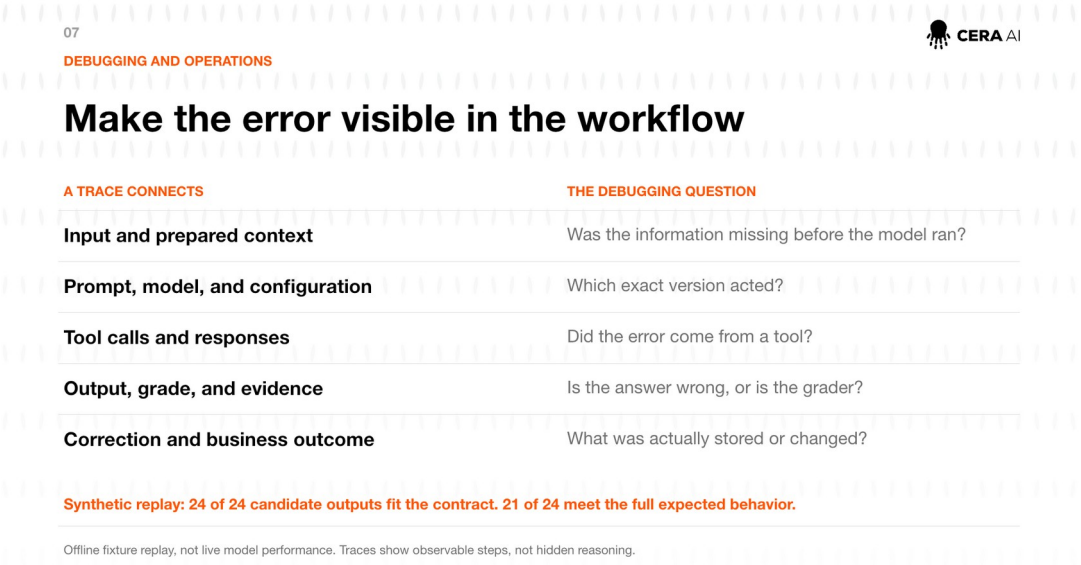
Using independent Beta(1,1) priors gives posteriors Beta(18,4) and Beta(19,3). Sampling the difference gives about a 67 percent probability that the candidate is better, but the 95 percent interval runs from roughly minus 16.8 to plus 26.1 percentage points. The interval crosses zero. That is inconclusive, not proof of equivalence and not evidence of a meaningful improvement.

For the workshop comparison, the repository also includes a scenario-aware hierarchical binary model. It fits each metric separately, checks convergence diagnostics, and repeats the fit with doubled prior scale. The model is only interpretable when both fits pass.

Protected rules stay outside the average. A single proposed payment action fails the protected-action check even if other scores improve.

Transition: Numbers tell us that something failed. A trace helps us locate where.

Make the error visible in the workflow



TIMING 22:00-25:30

Visual cue: Pair each item in the left column with its debugging question on the right.

A useful trace connects the observable steps around a result. Start with the input and the prepared context. Was the needed information missing before the model ran? Then record the exact prompt, model, and configuration. Which version actually acted? Next, keep tool requests and responses. Did a lookup return no result, or did the model ignore a correct result? Finally, connect the output to the grade, its evidence, and any real application change.

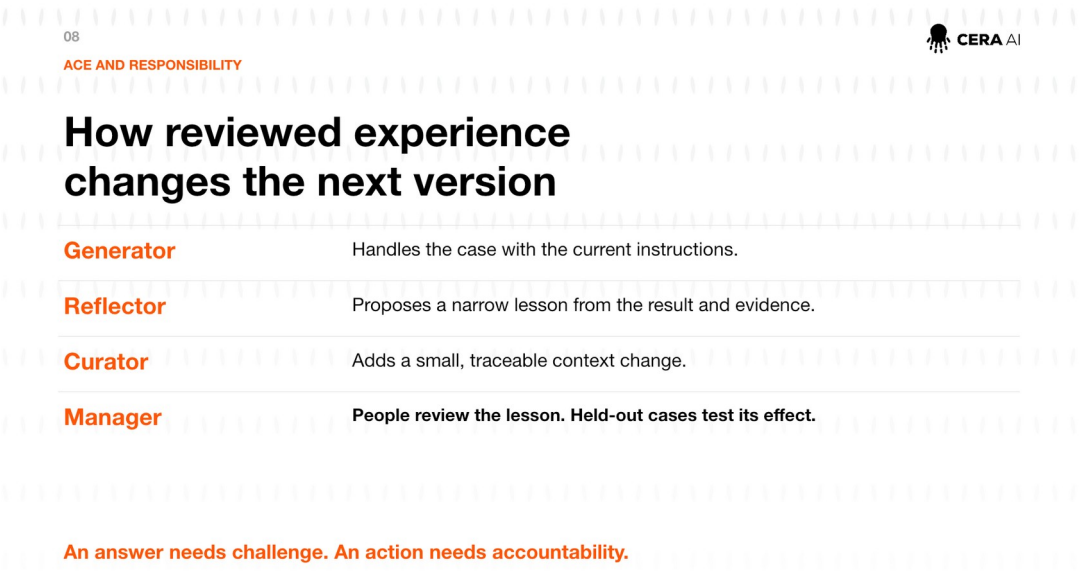
The orange box reports the synthetic replay. Eight cases repeated three times create 8 times 3, or 24 attempts for each version. Across two versions, that is 24 times 2, or 48 total attempts. The candidate fits the seven-key contract on 24 of 24 attempts. It meets the full expected behavior on 21 of 24 attempts. The missing three are the same authored failure repeated for one scenario.

Those numbers are ceilings within a fixed fixture, not averages from live traffic. The replay makes zero network calls. It proves that the runner, graders, and reporting path are reproducible; it does not estimate how a model will behave.

A trace also has a boundary. It shows recorded inputs, calls, outputs, and grades. It does not reveal a model's hidden reasoning, and we should not present it as if it did.

Transition: Once the failure is visible, the final question is how the organization learns without letting the system approve itself.

How reviewed experience changes the next version



TIMING 25:30-29:00

Visual cue: Move from Generator to Manager and pause on the final sentence.

A reviewed learning cycle keeps four roles distinct. The generator handles the case with the current instructions. The reflector reads the result and evidence, then proposes one narrow lesson. The curator turns that lesson into a small, traceable context change. The manager is the human process that reviews the lesson and decides whether it should become a candidate.

This resembles Agentic Context Engineering: improving the context an agent uses through small, inspectable changes rather than changing model weights. The important word is reviewed. A reflector's lesson is another generated output. It can overfit one case, conflict with another rule, or generalize too broadly.

The held-out cases therefore test the proposed change. Protected rules remain vetoes. The baseline remains available when evidence is inconclusive. A release decision also considers downstream consumers, rollback, cost, and the consequences of being wrong.

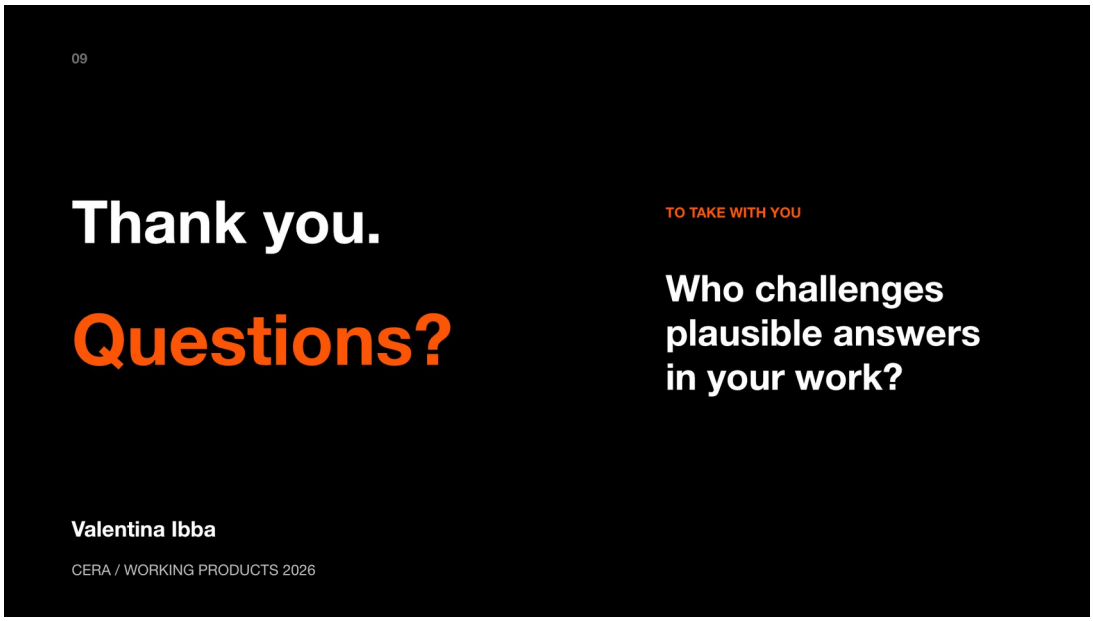
In the public workshop, nothing automatically rewrites the active prompt, calls a paid model, publishes a repository, or performs an external conference action. Those boundaries are intentional. The artifacts demonstrate how to prepare evidence, not how to remove accountability.

An answer needs challenge because fluent text can hide a wrong assumption. An action needs accountability because a technically valid action can still be unauthorized.

Transition: I will leave you with one question to take back to your own workflow.

SLIDE 9 OF 9

Thank you. Questions?



TIMING 29:00-30:00

Visual cue: Pause after the question and invite the audience in.

Thank you. The repository gives you a small synthetic example, the six-step workflow, deterministic checks, an offline replay, and the talk materials. You can replace the conference domain with your own only after defining the contract, permissions, and independent cases.

The question I want to leave with you is: who challenges plausible answers in your work? Is the challenge written down, repeatable, and connected to a decision? And when the agent proposes an action, who remains accountable for approving it?

Pause. Then ask: What questions would be useful to discuss?

If the room is quiet, offer this prompt: Where do you see errors that look convincing at first, and what evidence would make them visible?

> **Sources: the repository's docs/sources.md lists the public reading. All conference data and replay results shown here are synthetic. The 67 percent uncertainty example is labeled as a teaching illustration, not an operational result.**

Sources and honesty notes

- Anthropic, Building effective agents: https://www.anthropic.com/research/building-effective-agents
- Anthropic, Prompt engineering overview: https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/overview
- Gelman et al., Bayesian Data Analysis, third edition.
- Zhang et al., Agentic Context Engineering: https://arxiv.org/abs/2510.04618

Honesty notes

- The conference assistant, sessions, messages, outputs, and reported replay results are synthetic.
- The offline replay makes zero network calls and does not measure model variability.
- The 67 percent example is a separate teaching illustration using 17 of 20 versus 18 of 20.
- Passing finite cases does not prove general reliability. Protected-rule failures remain vetoes.
- The repository is local only. No publication or paid model call is part of this build.